Integrantes: Samanta Simba, Sebastián Cañizares, Jhostyn Paca
NRC: 36460
Fecha: 16/10/2026
Carrera: Ing. Electrónica y automatización
Grupo: 4

FUNDAMENTOS DE PROGRAMACIÓN · TEMAS 5, 6 Y 7

Guía gamificada: Misión Aventurero

Dos retos en C para practicar la estructura condicional compuesta (if-else) y el ciclo while.

Cómo jugar

1. Lee el reto y escribe tu código en C++ antes de mirar la solución.
2. Cada pista que uses descuenta 10 XP.
3. Prueba tu programa con todos los casos de prueba.
4. Suma tus XP y descubre tu rango.

2 retos · 100 XP cada uno · 200 XP en total

Rangos

Rango	XP	Nivel
Aprendiz	0 – 99 XP	Repasa el tema y vuelve a intentarlo
Aventurero	100 – 179 XP	Resolviste al menos un reto completo
Maestro del código	180 – 200 XP	Dominas ambas estructuras

Reto 1 · La tienda del aventurero

Estructura: if-else (condicional compuesta) | **Puntaje:** 100 XP

Enunciado

Una espada cuesta 50 monedas. Lee las monedas que tiene el jugador. Si le alcanzan, muestra "Compra exitosa: obtienes la espada" y las monedas que le quedan; si no, muestra cuántas monedas le faltan.

Requisitos y puntos

Requisito	XP
Declarar variables y leer el dato	20
Condición monedas >= precio	30
Bloque verdadero con dos mensajes	25
Bloque falso que calcula lo que falta	25
Total	**100**

Casos de prueba

Entrada	Salida esperada
80	Compra exitosa… restantes: 30
50	Compra exitosa… restantes: 0
20	Te faltan 30 monedas

Pistas (cada una descuenta 10 XP)

- Estructura condicional compuesta: if con su else, cada uno con llaves.
- Lo que falta es precio - monedas.

Solución · reto1.cpp

```
#include <stdio.h>

int main()
{
    int monedas, precio = 50;
    printf("Monedas disponibles: ");
    scanf("%d", &monedas);
    if (monedas >= precio)
    {
        printf("Compra exitosa: obtienes la espada\n");
        printf("Monedas restantes: %d\n", monedas - precio);
    }
    else
    {
        printf("Te faltan %d monedas\n", precio - monedas);
    }
    return 0;
}
```

Cómo funciona

- **Condición:** monedas >= precio es verdadera con 50 o más monedas; por eso 50 compra y deja 0.
- **Bloque verdadero:** Dos sentencias dentro de llaves: el mensaje de compra y las monedas restantes.
- **Bloque falso (else):** Se ejecuta solo si la condición es falsa y calcula precio - monedas.

Analisis del problema

Problema: Mediante este programa, analizaremos si el jugador cuenta con 50 o más monedas para obtener la espada.

Entrada: Ingresar el dato de las monedas

Proceso: Evaluar si el jugador tiene 50 o mas monedas: obtiene la espada, de no ser así, no obtiene la espada

Salida: Se mostrara un mensaje en pantalla indicando si obtuvo o no la espada

Tabla de valores

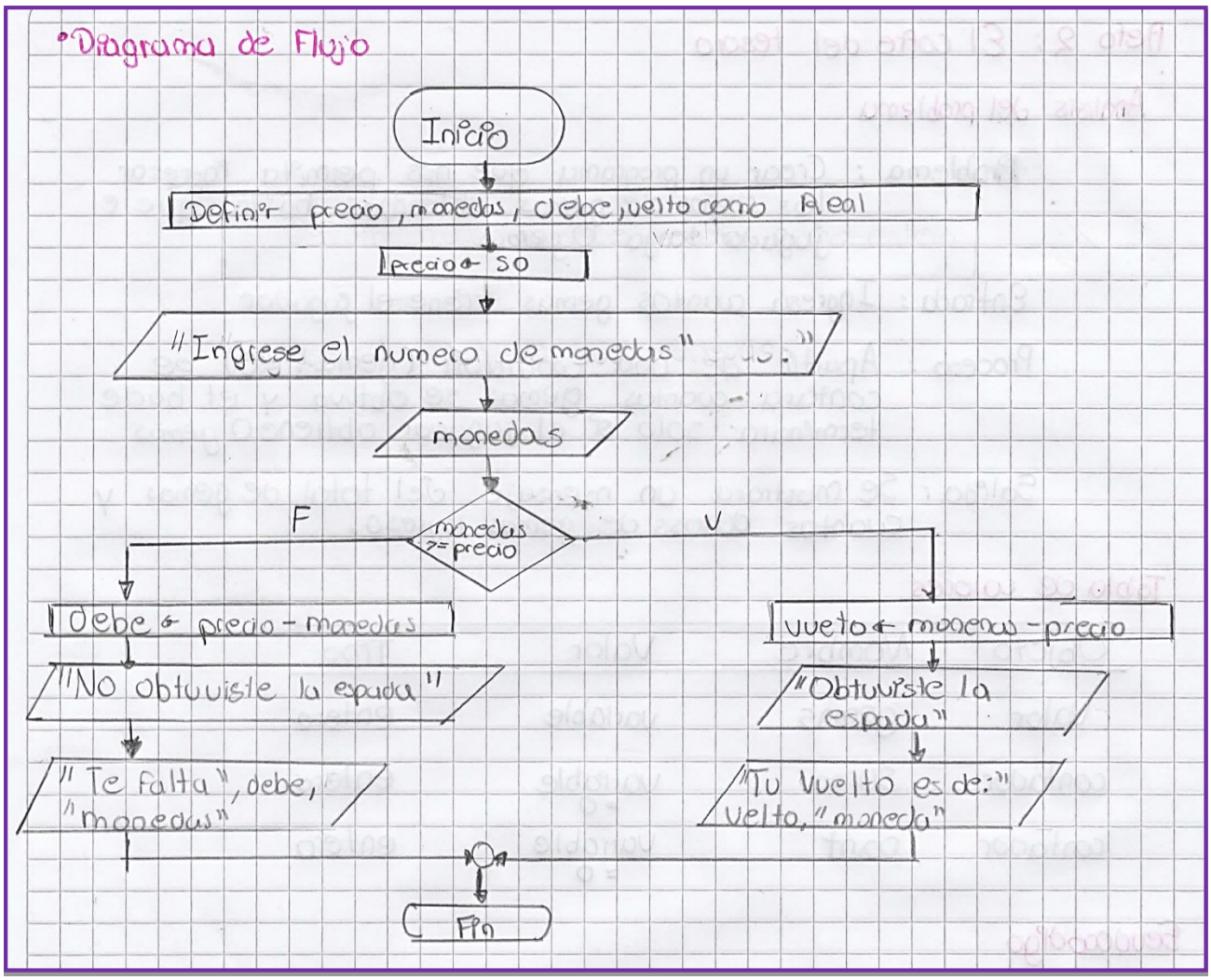
Objeto	Nombre	valor	Tipo
Valor	monedas	variable	entero
Valor	vuelto	variable	entero
Valor	debe	variable	entero
Valor	precio	constante = 50	entero

Pseudocodigo → Seudocodigo

```
Algoritmo laTienda-del-Aventurero
{
  Definir precio, monedas, debe, vuelto Como Real
  precio←50;
  escribir "Ingrese el numero de monedas";
  Leer monedas;
  Si monedas >= precio entonces
  {
    vuelto ← monedas - precio;
    escribir "obtuviste la espada"
    escribir "Tu vuelto es de:", vuelto, "monedas";
  }
  Si no
  {
    debe ← precio - monedas;
    escribir "No obtuviste la espada";
    escribir "Te faltan:", debe, "monedas";
  }
  FinSi
}
FinSi
```

Prueba de escritorio

Entrada	Proceso	Salida
moneda	monedas >= precio	mensaje
80	(verdadero) vuelto = 80-50 = 30	Obtuviste la espada Tu vuelto es de: 30 monedas
50	(verdadero) vuelto = 50-50 = 0	Obtuviste la espada Tu vuelto es de: 0 monedas
20	(Falso) debe = 50-20 = 30	No obtuviste la espada Te faltan: 30 monedas


•Diagrama de Flujo
Inicio
Definir precio, monedas, debe, vuelto como Real
precio ← 50
"Ingrese el numero de monedas"
monedas
monedas >= precio
F
V
debe ← precio - monedas
"No obtuviste la espada"
"Te falta", debe, "monedas"
vuelto ← monedas - precio
"Obtuviste la espada"
"Tu vuelto es de:" vuelto, "moneda"
Fin

→ Seudocodigo

```
Algoritmo LaTiendaAventurero
    Definir moneda, debe, vuelto, precio como entero;
    precio ← 50;
    Escribir "Ingrese el valor de sus monedas:";
    Leer moneda;
    Switch (moneda) entonces
        opcion1 (moneda < 50)
            debe = precio - moneda;
            Escribir "Debe", debe, "monedas";
        opcion2 (moneda = 50)
            Escribir "Obtuviste la espada";
        default
            vuelto ← moneda - precio;
            escribir "Obtuviste la espada y tu vuelto es de:", vuelto;
    FinSwitch
FinAlgoritmo
```

→ Prueba de escritorio

Entrada	Proceso	Salida
		Ingrese el valor de sus monedas:
80	default - vuelto 80-50 = 30	Obtuviste la espada y tu vuelto es de : 30
		Ingrese el valor de sus monedas:
50	opción 50=50 V	Obtuviste la espada
20	opción 20<50	Debe : 30 monedas
	debe = 50 - 20 = 30	

Reto 2 · El cofre del tesoro

Estructura: while | **Puntaje:** 100 XP

Enunciado

Lee el valor de cada gema que abre el jugador y repite hasta que ingrese 0. Al terminar muestra cuántas gemas abrió y la suma de sus valores.

Requisitos y puntos

Requisito	XP
Variables suma y cant iniciadas en 0	20
Lectura inicial antes del while	20
Condición gema != 0	20
Acumular, contar y releer dentro del ciclo	30
Mostrar los resultados	10
Total	**100**

Casos de prueba

Entrada	Salida esperada
`5, 10, 3, 0`	`Gemas: 3 · Total: 18`
`7, 0`	`Gemas: 1 · Total: 7`
`0`	`Gemas: 0 · Total: 0`

Pistas (cada una descuenta 10 XP)

- El 0 es el centinela: termina el ciclo y no se suma.
- Dentro del ciclo hay que volver a leer; si no, el bucle es infinito.

Solución · reto2.cpp

```
#include <stdio.h>

int main()
{
    int gema, suma = 0, cant = 0;
    printf("Valor de la gema (0 para terminar): ");
    scanf("%d", &gema);
    while (gema != 0)
    {
        suma = suma + gema;
        cant++;
        printf("Valor de la gema (0 para terminar): ");
        scanf("%d", &gema);
    }
    printf("Gemas: %d\n", cant);
    printf("Total: %d\n", suma);
    return 0;
}
```

Cómo funciona

- **Lectura inicial:** El while es pretest: primero se lee una gema y luego se decide si entra al ciclo.
- **Acumulador y contador:** suma acumula los valores y cant++ cuenta cuántas gemas se abrieron.
- **Relectura:** La última sentencia del cuerpo lee la siguiente gema; así el ciclo puede terminar con el 0.

Analisis del problema

Problema : Crear un programa que nos permita ingresar los datos de gemas obtenidos hasta que el jugador tenga 0 gemas

Entrada : Ingresa cuantas gemas tiene el jugador

Proceso : Apartir de una condicion (gema!=0) se contara cuantas gemas se obtuvo y el bucle terminara solo si el jugador obtiene 0 gemas

Salida : Se mostrara un mensaje del total de gemas y cuantos datos de gemas ingreso.

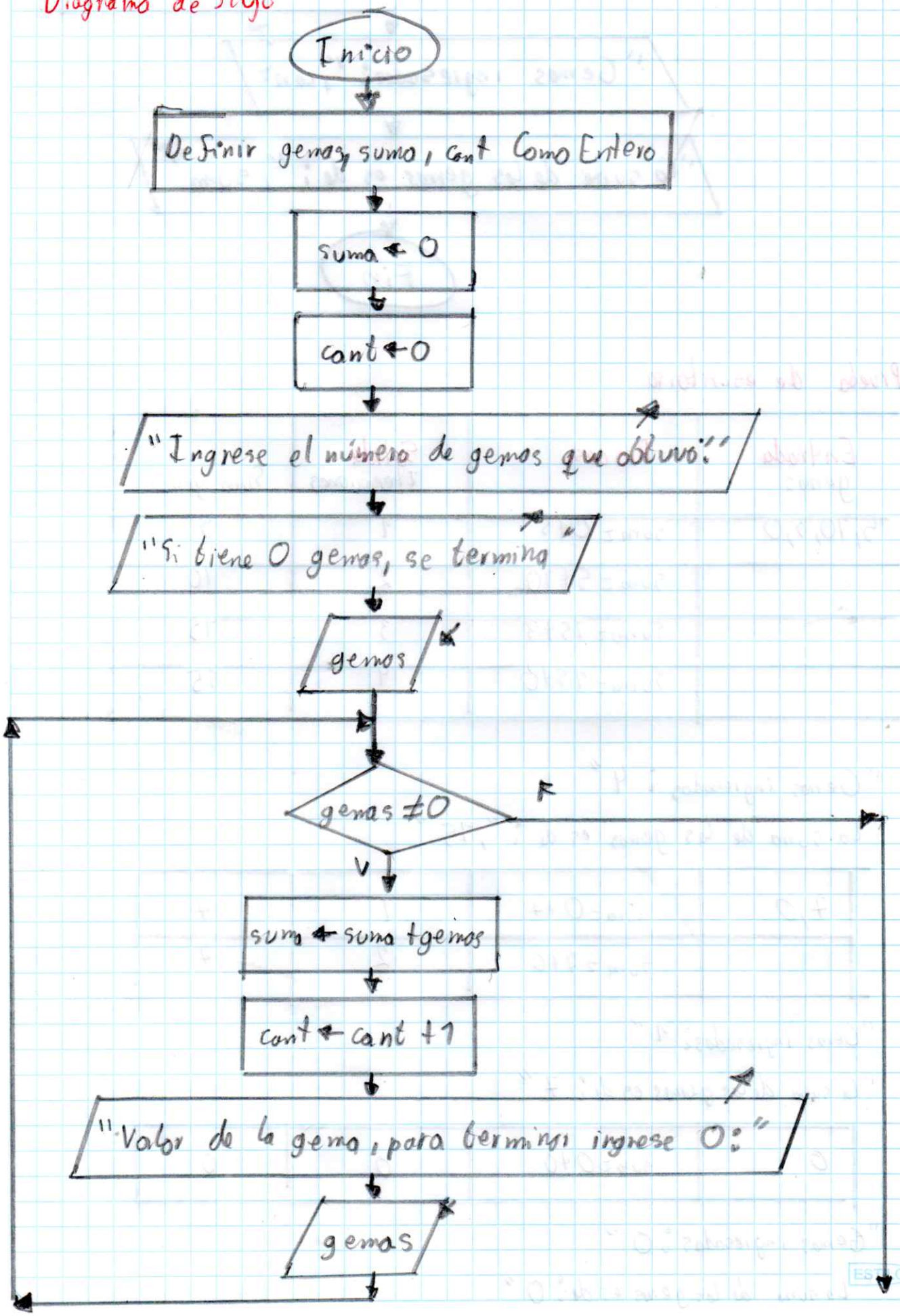
Tabla de valores

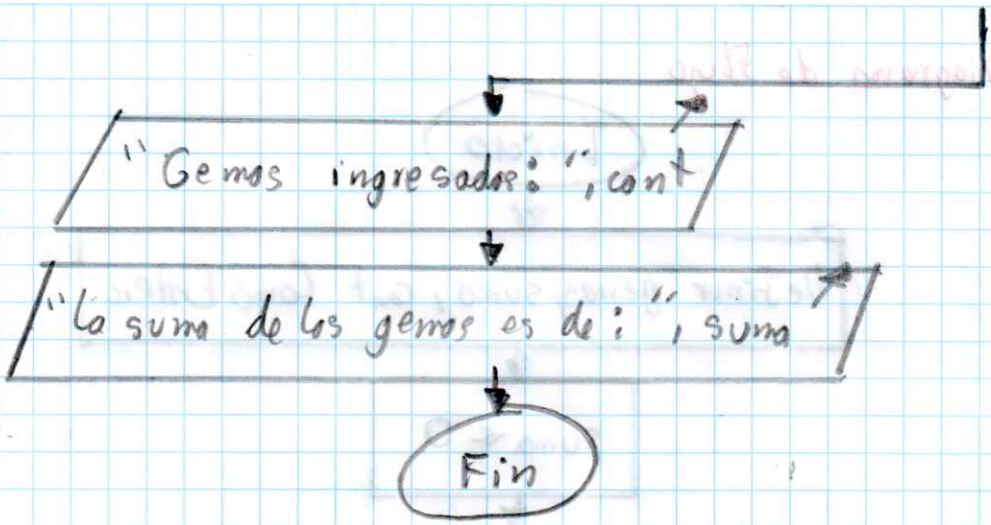
Objeto	Nombre	Valor	Tipo
Valor	gemas	variable	entero
contador	suma	variable = 0	entero
contador	cant	variable = 0	entero

Seudocodigo

```
Algoritmo Elcofre_del_Tesoro
    Definir gemas, suma, cant como entero;
    suma ← 0;
    cant ← 0;
    escribir "Ingrese el numero de gemas que obtuvo";
    escribir "Si tiene 0 gemas, se termina";
    Leer gemas;
    mientras gemas!=0 Hacer
        suma ← suma+gema;
        cant++
        escribir "Valor de la gema, para terminar ingrese 0.";
    Fin mientras
    escribir "Gemas ingresadas:" cant;
    escribir "La suma de las gemas es de:" suma;
Fin Algoritmo
```

Diagrama de Flujo





Prueba de escritorio

Entrada gemas	Proceso	Salida Iteraciones	Suma gemas
5, 10, 3, 0	suma = 0+5	1	5
	suma = 5+10	2	10
	suma = 15+3	3	15
	suma = 15+0	4	15

"Gemas ingresadas: 4"

"La suma de las gemas es de: ", 15"

7, 0	suma = 0+7	1	7
	suma = 7+0	2	7

"Gemas ingresadas: 1"

"La suma de las gemas es de: 7"

0	suma = 0+0	0	0

"Gemas ingresadas: 0"

"La suma de las gems es de: 0"

Rúbrica de pensamiento crítico

Después de resolver los retos, responde por escrito las cuatro preguntas. Cada una vale 5 puntos; el total es de 20 puntos. Las respuestas deben apoyarse en el código y en los casos de prueba de la guía.

Preguntas

1. **Análisis (5 pts).** En el reto 1 la condición es monedas >= precio. Si se cambiara por monedas > precio, ¿qué salida daría el caso de prueba con 50 monedas y por qué? ¿Cuál de las dos condiciones refleja mejor el enunciado?

   Si la condición cambia a monedas > precio sabiendo que precio es un valor fijo (50), daría como resultado que **no obtuvo la espada** (indicando que faltan monedas), ya que la condición estrictamente mayor (>) evalúa si el número es superior a 50. Al ingresar exactamente 50, la condición se vuelve falsa, ignorando que el jugador tiene el dinero exacto para pagar la espada. La condición monedas >= precio es la más correcta.

2. **Inferencia (5 pts).** En el reto 2, ¿qué pasaría con la entrada 5, 10, 3, 0 si se elimina el segundo scanf que está dentro del while? Predice el comportamiento, explica la causa y describe cómo lo detectarías al probar el programa.

   Se genera un **bucle infinito**. Al eliminar el segundo scanf dentro del while, el programa pierde la capacidad de actualizar el valor de la variable de entrada. Se quedará atascado evaluando eternamente el primer número ingresado (el 5), repitiendo la misma acción sin avanzar al 10, 3 o 0.

   Al realizar la prueba de escritorio o ejecutar el programa, nos daremos cuenta que la consola se queda congelada o repitiendo el mismo mensaje de manera indefinida, sin permitir introducir los siguientes datos.

3. **Evaluación (5 pts).** Si el reto 2 se escribiera con do-while en lugar de while, ¿qué cambiaría cuando la primera entrada es 0? Compara ambas estructuras con al menos dos criterios y decide cuál es más adecuada para este problema.

   Si la primera entrada es 0, un ciclo do-while ejecutaría el bloque de instrucciones al menos una vez antes de evaluar la condición final, lo que intentaría procesar una compra inválida de manera innecesaria.

   El while es más adecuado para este problema porque evalúa la condición al inicio. Si el usuario ingresa una condición de parada (como 0 monedas), el programa simplemente no entra al bucle, evitando errores.

4. **Creación y argumentación (5 pts).** Propón una mejora del juego que combine ambos retos: por ejemplo, una tienda que repite compras mientras al jugador le queden monedas. Describe o escribe la estructura (qué va dentro de qué), justifica por qué conviene anidarla y menciona dos precauciones que tomarías.

   Se puede implementar un ciclo repetitivo principal que mantenga el juego activo mientras al jugador le alcancen las monedas para comprar. Dentro de este ciclo se anida una estructura condicional que procesa cada transacción restando el costo y actualizando el saldo, tomando como precauciones actualizar correctamente la variable de las monedas en cada iteración para evitar bucles infinitos y validar que los valores ingresados no sean negativos.

Matriz de evaluación

Criterio	Excelente (5)	Satisfactorio (4)	En desarrollo (2–3)	Insuficiente (0–1)
P1 · Análisis	Compara >= con > usando el caso de 50 monedas, indica la salida exacta de cada versión y decide cuál es correcta con base en el enunciado.	Identifica la diferencia en el caso de 50 monedas y elige la condición correcta, con una justificación incompleta.	Menciona que las condiciones difieren pero no relaciona la diferencia con el caso de prueba o la salida es parcialmente errónea.	No identifica la diferencia o la salida indicada es incorrecta.
P2 · Inferencia	Predice que el bucle no termina, explica que gema no cambia dentro del ciclo y propone una forma concreta de detectarlo (caso de prueba, traza o contador de vueltas).	Predice el bucle infinito y explica la causa, pero la forma de detectarlo es vaga.	Intuye que hay un problema, pero no explica la causa o predice un resultado incorrecto.	No predice el comportamiento o lo describe de forma incorrecta.
P3 · Evaluación	Explica qué ocurre con la entrada 0 en cada estructura, compara con dos o más criterios (momento de evaluar la condición, mínimo de ejecuciones, necesidad de validar) y concluye con un argumento sólido.	Explica el efecto de la entrada 0 y compara con un criterio, con una conclusión razonable.	Describe cada estructura por separado sin compararlas o su conclusión no se apoya en criterios.	Confunde las estructuras o no emite una conclusión.
P4 · Creación y argumentación	Propone una estructura anidada viable y bien ordenada, justifica por qué anidar y menciona dos precauciones pertinentes (por ejemplo, evitar bucles infinitos y probar varios casos).	Propone una estructura viable y justifica el anidamiento; menciona una precaución.	La propuesta es poco clara o incompleta, o la justificación y las precauciones son genéricas.	No propone una estructura o la propuesta no funciona.

Hoja de calificación

Pregunta	Puntaje	Observaciones
P1 · Análisis	____ / 5	
P2 · Inferencia	____ / 5	
P3 · Evaluación	____ / 5	
P4 · Creación y argumentación	____ / 5	
Total	**____ / 20**	